

Plan & Feasibility Study: Proxmox Automated Testing Environment

This document assesses the feasibility and details the design for creating a Proxmox-based testing skill. This skill will automate the cloning of a virtual machine template, deployment of PiSelfhosting components, verification of their functionality, and subsequent teardown of the VM.

1. Feasibility Assessment

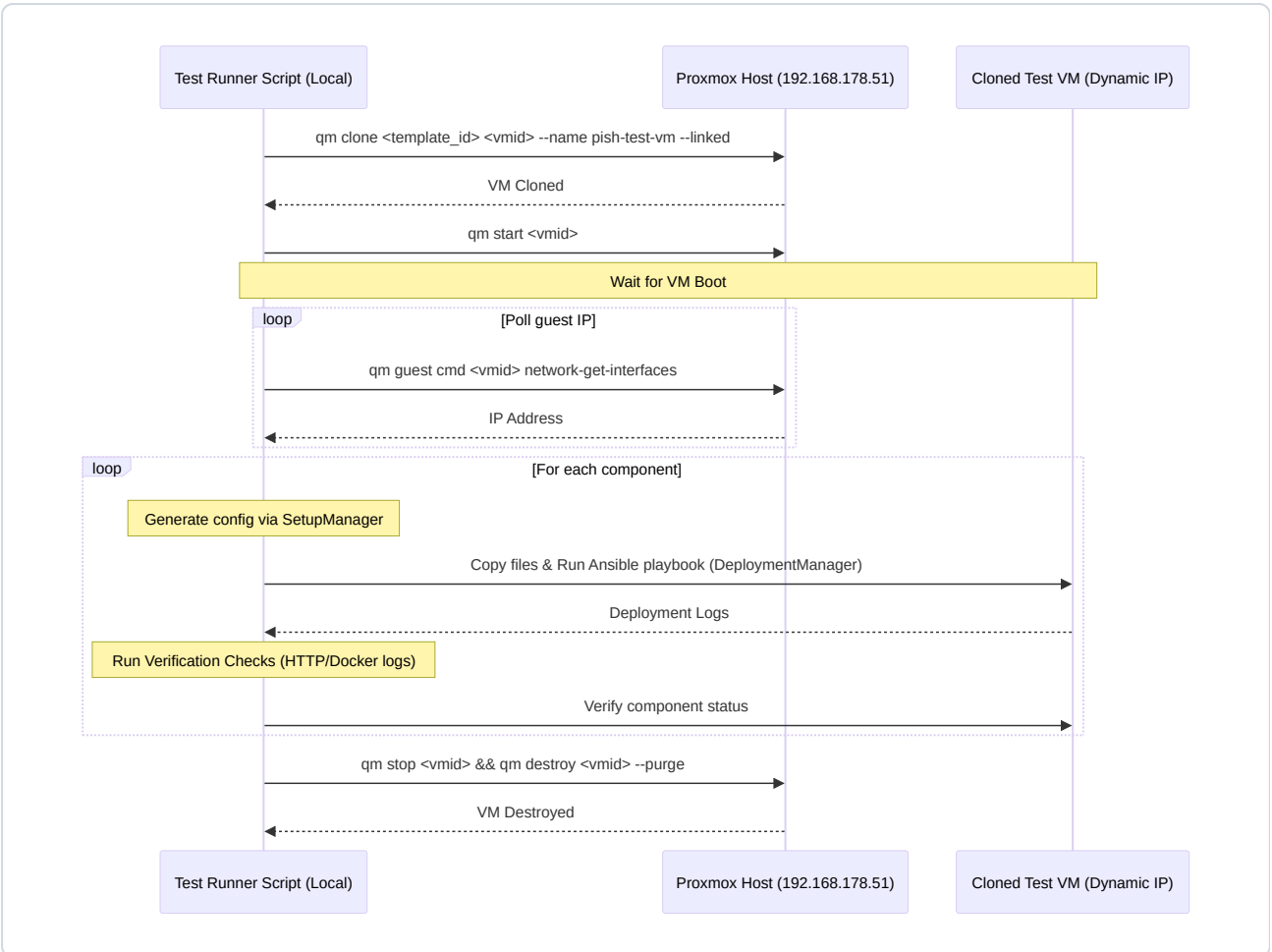
Conclusion: Highly Feasible and Architecturally Sound.

Integrating a Proxmox testing flow is extremely viable and provides a high-fidelity testing method. Key assessment points:

- 1. Reusability of Core Code:** We can directly import and reuse [SetupManager](#) and [DeploymentManager](#). This ensures that we test the *exact same* deployment pipeline and configuration files used by the production Flask app.
 - 2. Proxmox VM Lifecycle:** Proxmox VE offers a robust CLI (`qm`) that is easily accessible via SSH or REST API. Since the controller machine has SSH capabilities, executing commands like `qm clone` , `qm start` , and `qm destroy` via SSH to the Proxmox host (`192.168.178.51`) is simple, secure, and requires no additional dependencies.
 - 3. VM IP Discovery:** If the master template has the `qemu-guest-agent` installed, we can query its IP address using `qm guest cmd <vmid> network-get-interfaces` . Alternatively, we can use Proxmox's Cloud-Init integration to set a static IP before booting the VM.
 - 4. Execution Time:** Deploying and testing all ~38 components sequentially will take significant time (approx. 1 to 2 minutes per component, totaling 40–80 minutes). Because this is a background process, we can run it as an asynchronous task using the `/goal` slash command.
-

2. Architecture & Workflow

Here is how the automated test runner orchestrates the Proxmox host and the test VM:



3. Implementation Details

A. Proxmox VM Control Commands

Using SSH commands targeting `root@192.168.178.51` :

1. Find template VM ID:

```
qm list | grep pi-server-vm
```

2. Clone VM (Linked clone for extreme speed and low disk usage):

```
qm clone <template_vmid> <test_vmid> --name pish-test-vm --linked
```

3. Start VM:

```
qm start <test_vmid>
```

4. Wait and Get IP Address:

```
qm guest cmd <test_vmid> network-get-interfaces
```

5. Teardown:

```
qm stop <test_vmid> && qm destroy <test_vmid> --purge
```

B. Integrating the Managers

We will create a test runner script (e.g., `scripts/proxmox_test_runner.py`) which acts as the orchestrator:

```
from pathlib import Path
from src.managers.component_reader import ComponentReader
from src.managers.setup_manager import SetupManager
from src.managers.deployment_manager import DeploymentManager

# Initialize managers
reader = ComponentReader(metadata_path="config/components_metadata.json")
setup_mgr = SetupManager(component_manager=reader, output_dir=Path("./tmp_proxmox"))
deploy_mgr = DeploymentManager(component_manager=reader)

# Example sequence to prepare and deploy a component
setup_mgr.prepare_deployment_package(
    selected_components=["adguard-home"],
    user_variables={"PISelfhosting_HOST_IP": vm_ip},
    managed_devices=[{"ip": vm_ip, "username": "hvhoek", "password": "..."}]
)
```

C. Component Verification Engine

For each component deployed, the verification script will perform: 1. **Container**

Check: Verify the container is running and healthy: `bash ssh hvhoek@<vm_ip> "docker ps --filter name=piselfhosting-<component_id> --format '{{.Status}}'"` 2. **Log**

Scan: Check for common error signatures (e.g., `Traceback`, `Exception`, `Fatal`) in logs: `bash ssh hvhoek@<vm_ip> "docker logs piselfhosting-<component_id> 2>&1"` 3.

HTTP Port Check: If `has_ui: true`, make an HTTP request to `http://<vm_ip>:<port>` and assert success.

4. Key Performance Optimizations

To keep the run duration manageable, we can apply the following strategies: -

Linked Clones: Cloning the VM takes under 2 seconds. - **Docker Cache Mirroring:** If the VM template is pre-configured to use a local registry mirror or a docker registry pull-through cache, image downloading will be super fast and won't strain internet bandwidth. - **Incremental Cleanup vs. Fresh Installs:** - We can reuse a single VM instance for multiple components that do not conflict, restarting the VM or performing docker volume pruning between components. - Or, to guarantee absolute isolation, we can clone a new VM, test a component, and destroy it. The script can support both modes via a `--fresh-vm-per-component` flag.

5. Skill Proposal: `proxmox-test`

We can add a new skill to the project: [SKILL.md](#) containing the workflows, parameters, and troubleshooting guides for Proxmox-based automated component verification.

Target Skill Outline:

- **Location:** `.agents/skills/proxmox-test/SKILL.md`
- **Contents:**
 - Configuration instructions (`.env` variables for Proxmox server credentials, master template ID).
 - CLI usage examples to run the tests.
 - How to interpret the resulting test report (`docs/PROXMOX_TESTS.md`).